Aalto University
School of Arts, Design
and Architecture

Check point no.6
Topic: Distributed Mixture-of-Experts (MoE) systems

Phuong Ngo Pitkänen and Amirreza Jafariandehkordi

The inefficiency of existing MoE training methods mainly comes from dynamic expert selection and flexible MoE structure.

- Skewed expert selection leads to dynamic load imbalance.
- Synchronous execution mode of operations is inefficient.
- Expert parallelism causes severe network contention.

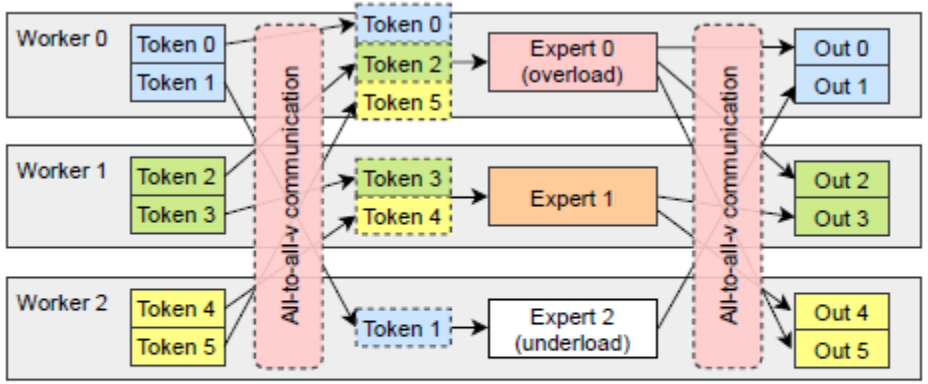
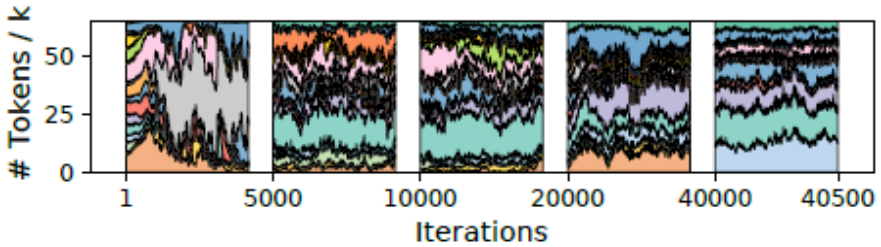
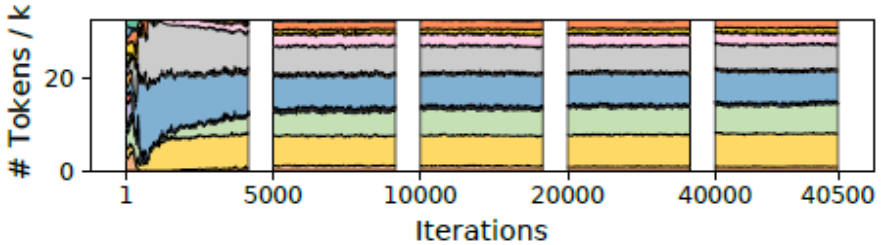


Figure 3. Partitioning of tensors in expert parallelism, with related communication.



(a) Layer 12 of MoE-BERT-Deep.



(b) Layer 8 of MoE-GPT.

Areas with different colors represent different experts.

Figure 4. Distribution of expert selection in some iterations when training different MoE models.

Source: He, J., Zhai, J., Antunes, T., Wang, H., Luo, F., Shi, S., & Li, Q. (2022). FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models. *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '22)*, 120–132. https://doi.org/10.1145/3503221.3508418.

FasterMoE: performance modelling

Load-aware Computation Modeling

When tokens are routed to experts, some GPUs end up with far more work than others. The model **predicts how long the slowest GPU will take to finish**, since everyone has to wait for it before moving on. The more uneven the token distribution, the longer this wait becomes.

Topology-aware Communication Modeling

After computation, GPUs need to send results back across the network. The model **finds whichever network link is carrying the most traffic** and uses that as the predicted bottleneck. It accounts for both the connections inside a single node and the slower links between nodes.

DDL-Roofline Model

This combines the two models above to show whether your training is **stuck waiting on computation or communication**. If computation dominates, you should fix load imbalance; if communication dominates, you should reduce traffic. It gives a theoretical ceiling so you know how much room for improvement exists.

Source: He, J., Zhai, J., Antunes, T., Wang, H., Luo, F., Shi, S., & Li, Q. (2022). FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models. *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '22)*, 120–132. https://doi.org/10.1145/3503221.3508418

Switch Transformers: auxilliary load balancing loss and capacity factor

A hard cap on how many tokens pile up on the same experts, nudging the router to spread more evenly.

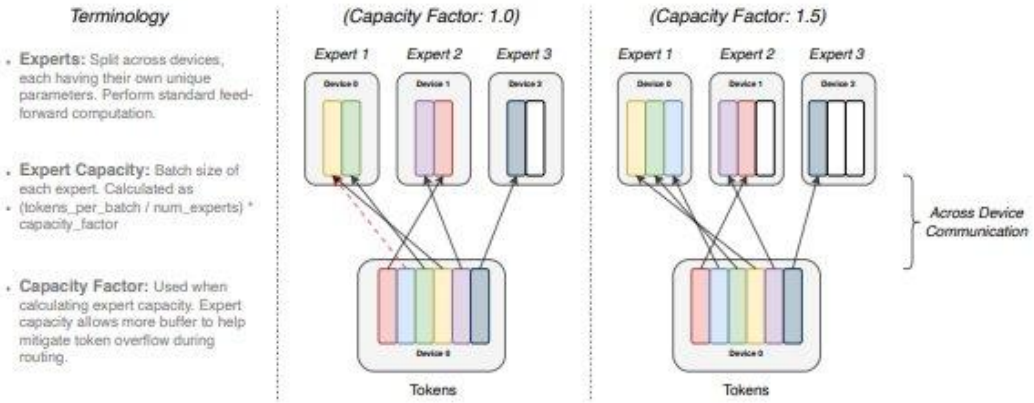


Figure 3: Illustration of token routing dynamics. Each expert processes a fixed batch-size of tokens modulated by the *capacity factor*. Each token is routed to the expert with the highest router probability, but each expert has a fixed batch size of (total_tokens / num_experts) × capacity_factor. If the tokens are unevenly dispatched then certain experts will overflow (denoted by dotted red lines), resulting in these tokens not being processed by this layer. A larger capacity factor alleviates this overflow issue, but also increases computation and communication costs (depicted by padded white/empty slots).

The loss penalizes the model whenever tokens pile up on the same experts.

A Differentiable Load Balancing Loss. To encourage a balanced load across experts we add an auxiliary loss (Shazeer et al., 2017, 2018; Lepikhin et al., 2020). As in Shazeer et al. (2018); Lepikhin et al. (2020), Switch Transformers simplifies the original design in Shazeer et al. (2017) which had separate load-balancing and importance-weighting losses. For each Switch layer, this auxiliary loss is added to the total model loss during training. Given N experts indexed by $i = 1$ to N and a batch $\mathcal{B}$ with T tokens, the auxiliary loss is computed as the scaled dot-product between vectors f and P,

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i \tag{4}$$

where f_i is the fraction of tokens dispatched to expert i,

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax}\, p(x) = i\} \tag{5}$$

and P_i is the fraction of the router probability allocated for expert i, [2]

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x). \tag{6}$$

Since we seek uniform routing of the batch of tokens across the N experts, we desire both vectors to have values of $1/N$. The auxiliary loss of Equation 4 encourages uniform routing since it is minimized under a uniform distribution. The objective can also be differentiated as

Source: Fedus, Zoph & Shazeer (2022) *Switch Transformers*, §2.2 "Distributed Switch Implementation." arXiv:2101.03961, JMLR 23(120). CC-BY 4.0.

SmartMoE: Workload-aware Performance Modelling

- SmartMoE uses historical routing data to predict which experts will be overloaded in upcoming steps, then physically moves those experts to different GPUs to co-locate hot and cold experts on the same device.
- The cost model underneath this is a load prediction function, which estimates future token counts per expert using a moving window over past routing distributions, then solves an assignment problem to find the placement that minimizes the maximum load on any single GPU.

Source: Zhai, M., He, J., Ma, Z., Zong, Z., Zhang, R., & Zhai, J. (2023). SmartMoE: Efficiently Training Sparsely-Activated Models through Combining Offline and Online Parallelization. Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC 2023), 961–975.

Cost model implementation: Load-aware computation modeling

Core cost model and math

$$F = 4 \cdot B_{\text{total}} \cdot \alpha \cdot H^2 \cdot L$$

$$\text{lat}_{\text{comp}} = \frac{4 \cdot B_{max} \cdot \alpha \cdot H^2 \cdot L}{P_w}$$

$$P_w = \text{median}\left\{\frac{4 \cdot B_{\text{total}} \cdot \alpha \cdot H^2 \cdot L}{t_{\text{fwd}}}\right\} \dashv \}$$

B_{total} : total token-expert assignments in this step

$\alpha = 4$: the FFN expansion ratio (hardcoded, matches standard transformer FFN width)

L : number of layers

$H = 512$: hidden dimension

P_w : effective compute throughput, in FLOPs/sec

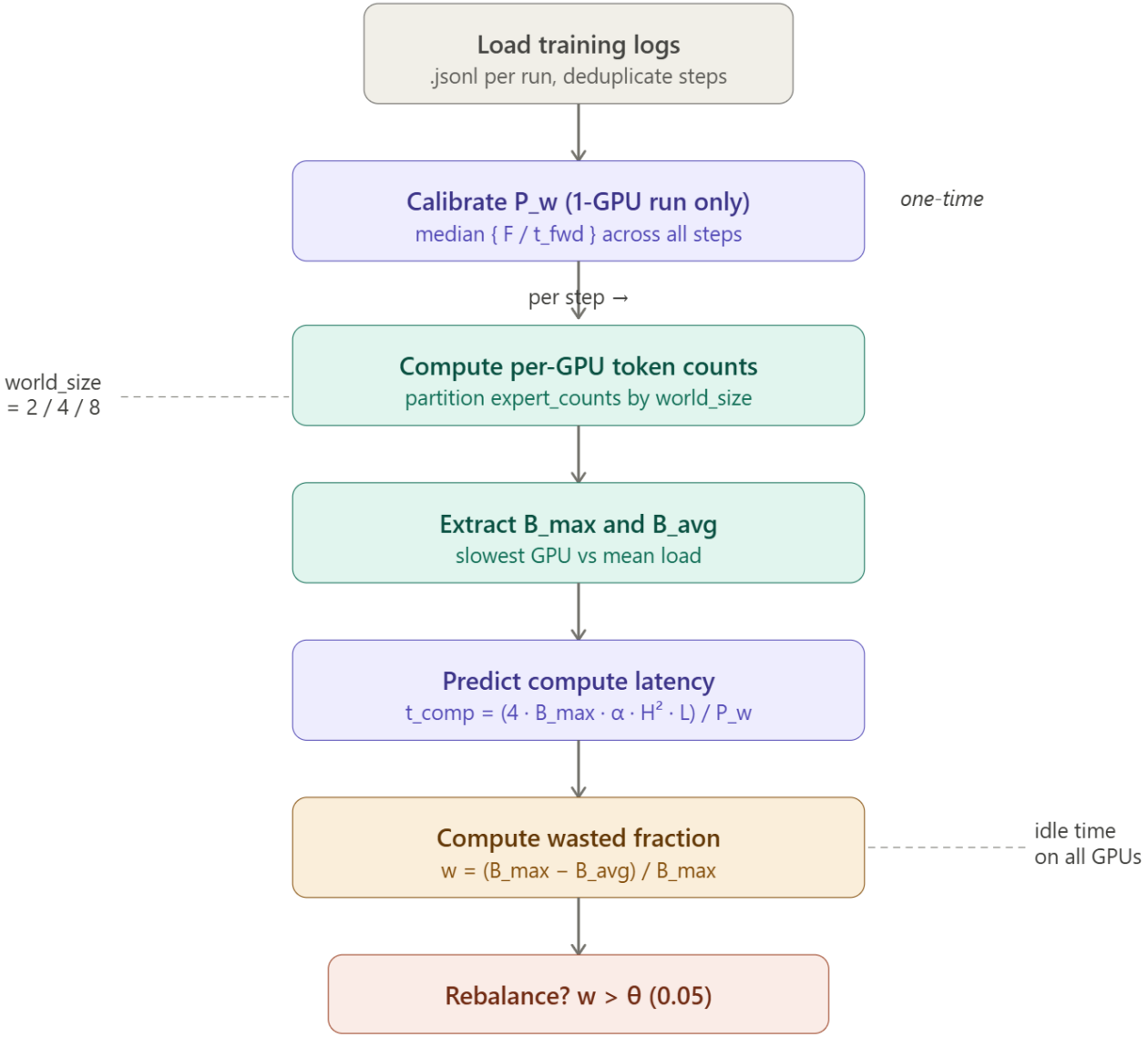
The core idea is to track the slowest GPU at every step and quantify how much compute the rest of the cluster is wasting waiting for it.

Source: Fedus, Zoph & Shazeer (2022) *Switch Transformers*, §2.2 "Distributed Switch Implementation." arXiv:2101.03961, JMLR 23(120). CC-BY 4.0.

Cost model implementation: Load-aware computation modeling

Implementation process

We start by loading the training logs, then calibrate the model's effective throughput once using the 1-GPU run. From there, at every step, we partition the expert assignments across GPUs, identify the slowest one, estimate the wasted compute time, and trigger a rebalance flag when the waste exceeds the threshold



Cost model implementation: Load-aware computation modeling

Design choice

1. Wasted fraction as the imbalance signal

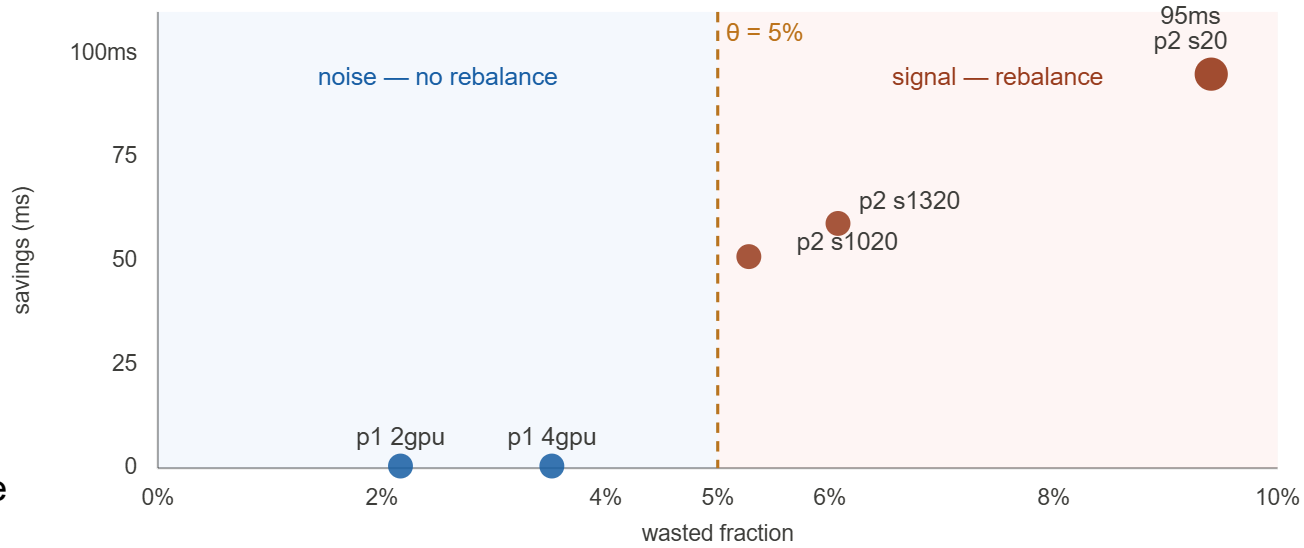
- Measures idle time as a share of the slowest GPU's work
- Architecture-agnostic — doesn't change with model size or batch size
- Directly maps to real lost compute, not a proxy metric

2. Calibrate P_w from 1-GPU run only

- Isolates pure compute cost, free of communication overhead
- Absorbs all real-world inefficiencies implicitly (memory bottlenecks, dispatch overhead)
- Transfers cleanly across world sizes as long as architecture stays the same

3. Fixed 5% threshold

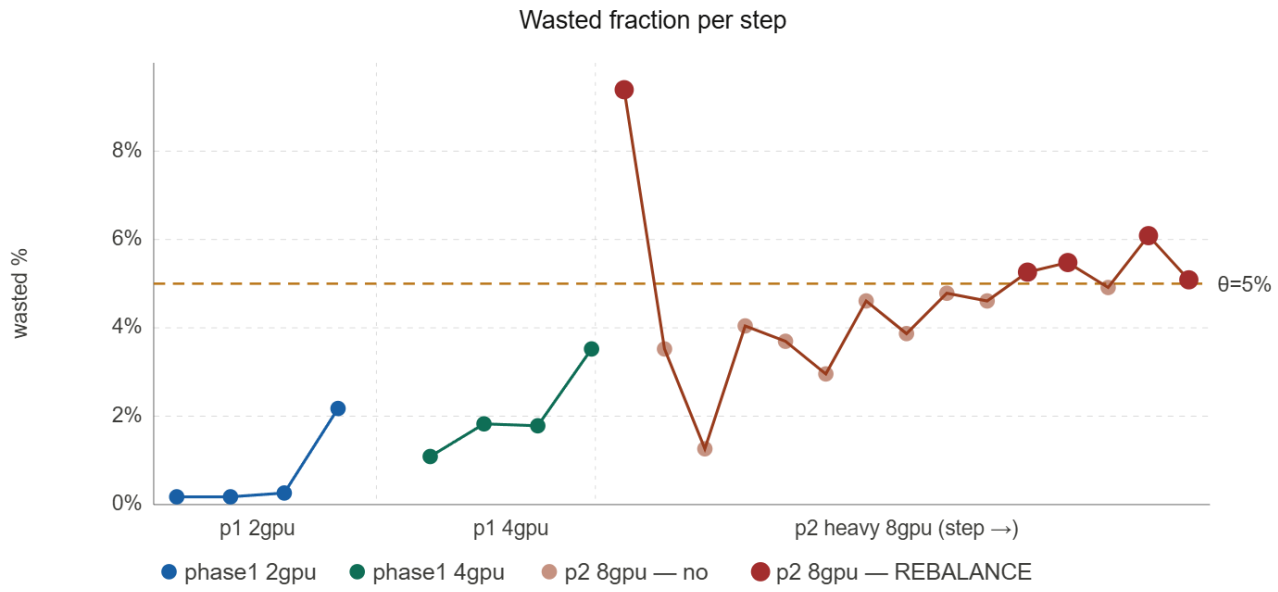
- Below 5%: savings are sub-millisecond, not worth rebalance overhead
- Above 5%: savings become meaningful (tens to hundreds of ms at scale)
- Known weakness: should ideally be relative to step time, not absolute waste fraction



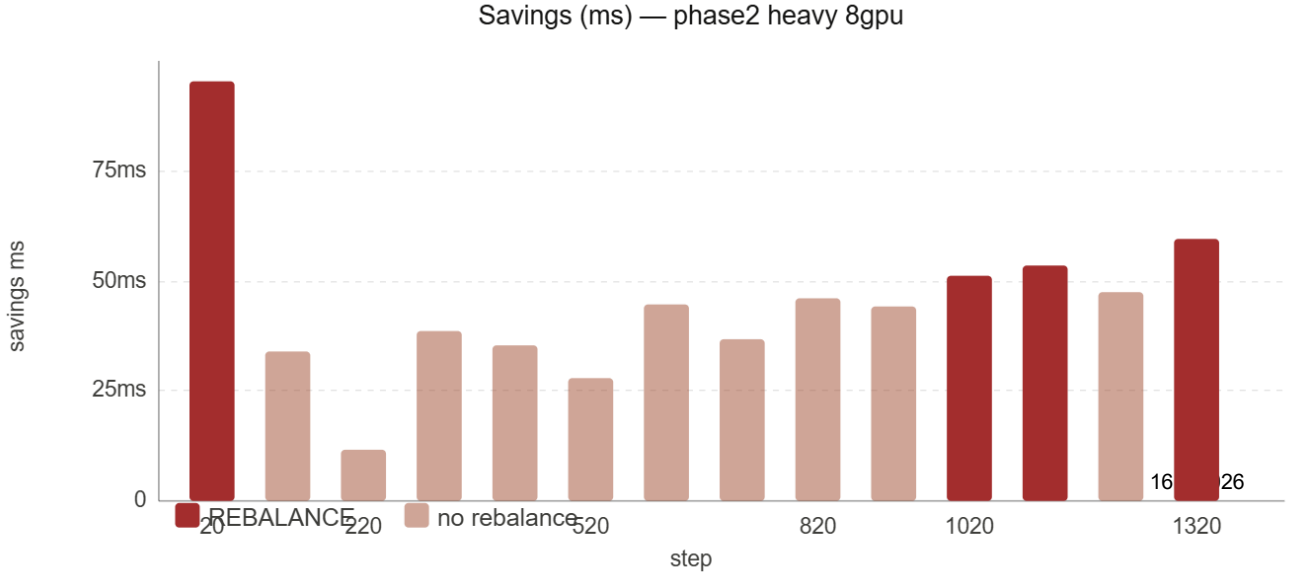
Cost model implementation: Load-aware computation modeling

Quick look at the results

All three runs against the 5% threshold — phase1 stays flat and quiet, while phase2 8gpu fluctuates persistently around the line.



The savings in ms for phase2 8gpu, making the step-20 outlier (95ms) and the late-training cluster of triggers (steps 1020–1420) immediately visible.



A!

A!

Kiitos
aalto.fi